My Class Notes: Software Development Processes, Quality, and Planning

1. SDLC & The Quality Umbrella

- **The Quality Umbrella Frameworks:** Quality management connects enterprise governance down to the individual developer using three main levels:
  - **CMMI:** Focused on organizational capacity, institutionalized processes, and enterprise-level quality assurance.
  - **TSP (Team Software Process):** Focused on team-level coordination, enabling self-directed teams to deliver quality software within budget and schedule constraints.
  - **PSP (Personal Software Process):** Focused on the individual engineer through disciplined estimation, defect logging, and personal measurement.
- **SDLC Integration:** SDLC models (like comparing Waterfall's rigid milestones to Agile's iterative feedback loops) establish the overarching discipline and lifecycle stages (requirements, design, coding, verification) used while executing PSP.
- **Core Takeaway:** Adopting structured, disciplined process frameworks consistently leads to measurable improvements in software quality and individual productivity.

2. PSP vs. TSP Summary

Dimension	Personal Software Process (PSP)	Team Software Process (TSP)
Focus Level	Individual software engineer	Collaborative engineering team
Primary Objective	Track and improve personal skills, work habits, productivity, and defect rates	Plan, assign, monitor, communicate, and deliver collective work
Data & Methodology	Strict data logging: metric-driven tracking of defects, time, and tasks	Defined team roles, shared plans, peer reviews, and milestone synchronization
Key Advantages	Precise personal estimation of time/effort; clear detection of personal error patterns	Effective teamwork, scalable project coordination, and collective risk mitigation
Limitations	Time-consuming data logging and administrative overhead	Added planning overhead, regular meetings, and need to adhere to team standards

Key Components of TSP

- **Team Planning:** Establishing a shared understanding of project requirements across all members.
- **Defined Roles & Responsibilities:** Assigning explicit ownership, performance goals, and individual duties.
- **Quality Management:** Conducting structured peer reviews and systematically tracking defects to minimize technical debt.
- **Progress Tracking:** Continuously comparing actual progress and metrics against planned targets.
- **Risk Management:** Identifying vulnerabilities early and implementing proactive mitigation plans.
- **Continuous Improvement:** Evaluating project metrics and lessons learned to refine future cycles.

3. Product Planning, Measurement & Time Management

Time Management & Job Logging

- **Task Categorization:** Decompose high-level deliverables into clear, categorized tasks with designated units of measure. Minutes are typically used as the standard measurement unit.
- **Interruption Tracking:** Explicitly log interruptions as distinct variable events to separate active development time from context switching.
- **Job Log Attributes:** A comprehensive job log tracks:
    - **Activity ID:** Task number, entry date, and process/lifecycle phase.
    - **Estimates:** Estimated task duration (min, max, and cumulative projections).
    - **Actual Performance:** Recorded execution time and observed rate of units delivered.
    - **To-Date Metrics:** Cumulative time spent, to-date unit completion rates, and historical min-max variance.

Size Measurement & Lines of Code (LOC)

- **Historical Baseline:** Compare planned code size against historical baseline data ("what was done before") to ground estimates in reality.
- **LOC Counting Standard:** Exclude blank lines and comment-only lines to measure true logical size.
- **Judgment & Consistency:** Maintain uniform measurement standards across projects, using sound engineering judgment when selecting size metrics.

4. Commitments & Professional Reputation

In software engineering, a commitment acts as a professional covenant binding technical estimates to real-world execution.

- **Core Dimensions:**
    - **What:** Scope and specifications of the exact deliverable.
    - **Why:** Business context, technical necessity, and underlying objectives.
    - **For Whom:** The client, end-user, or internal stakeholder receiving the output.
- **Types of Commitments:**

- **Contractual Commitment:** Formal obligation to deliver work in exchange for financial compensation, contract fulfillment, or rewards.
- **Personal Commitment:** Authentic, voluntary agreement driven by professional pride and ownership.
- **Full Agreement:** The ideal state when contractual obligations and personal responsibility fully synchronize.

Disciplined Commitment Management

1. **Analyze Before Committing:** Conduct thorough technical analysis and feasibility checks before agreeing to deadlines or scope.
2. **Make Responsibly:** Ensure promises are realistic, bounded by data, and actively managed.
3. **Notify Early:** Communicate immediately to stakeholders as soon as it's clear a deadline cannot be met.
4. **Track Progress:** Continuously monitor actual burn rates and milestones against agreed targets.

Consequences of Mismanaged Commitments

- **Misplaced Priorities:** Scrambling to meet impossible deadlines leads to misallocated resources and skipped process steps.
- **Degraded Quality:** Rushed code creates critical architectural flaws, high defect rates, and technical debt.
- **Loss of Trust:** Repeatedly missing delivery dates destroys stakeholder, managerial, and peer confidence.
- **Diminished Reputation:** Chronic inaccuracy damages your professional credibility and respect for your engineering judgment.

Marian Jemal

Section 2